Neighborhood Stability in Assignments on Graphs

A report on the paper by Aziz, Lisowski, Suzuki and Vollen (arXiv:2407.05240, 2024)

Ashik Stenny (230224) Achyuth H Warrier (230052)

CS698I: Relational Structures in Games, IIT Kanpur Instructor: Sunil Simon

14 November 2026

Abstract

This report is about the paper *Neighborhood Stability in Assignments on Graphs* by Aziz, Lisowski, Suzuki and Vollen. In the paper, agents are placed on the vertices of a seat graph. Each agent likes (approves) some of the other agents, and its utility is the number of liked agents sitting next to it. An assignment is *neighbourhood stable* if no two agents sitting next to each other would both be better off by swapping seats. The paper shows that such an assignment does not always exist (the counterexample uses complete bipartite seat graphs), and that letting agents swap can go on forever even on a cycle of 4 seats. On the positive side, a neighbourhood stable assignment always exists on cycles and paths, and can be found in polynomial time. For general seat graphs, one exists when the seat graph has enough leaves compared with the directed feedback vertex set number of the preference graph, and it can be found in polynomial time when the preference graph has no directed cycles. We go through each result using one small running example and connect the paper to topics from our course (potential games, the finite improvement property, PLS, colouring games and hedonic games). We also add a few observations of our own: a full description of when envy happens in the counterexample, an $O(n^2)$ running time for the path algorithm instead of $O(n^3)$, a five-agent example showing why distance-2 stability can fail on cycles, and two cases that the written proof of the main lemma for cycles does not handle, with fixes that we tested on a computer.

Contents

1 Introduction and motivation **3**

2 Model **4**

2.1 Running example 4

2.2 How the notions relate 4

2.3 Why binary preferences 5

3 NS assignments do not always exist **6**

4 Swapping can go on forever **8**

5 Cycle seat graphs **9**

5.1 The swap rule 9

5.2 Splitting into two cases 9

5.3 Case 1: a greedy seating 9

5.4 Case 2: path partitions 10

5.5 Why distance-2 stability fails on cycles [ours] 12

6 Path seat graphs **13**

7 A condition for general seat graphs **14**

8 Things we noticed [ours] **15**

8.1 Two cases the proof of Lemma 2 does not cover 15

8.2 Theorem 4: what is stated vs. what is proved 15

8.3 Running time of the path algorithm 16

8.4 Who actually finds the seating? 16

9 Links to the course **17**

10 Open questions and conclusion **18**

A Computer checks **19**

What is ours. Anything marked **[ours]** is our own work and not from the paper. Everything else is from the paper. The computer checks we mention are described in Appendix A.

1 Introduction and motivation

The dinner party. Suppose a host has to seat n guests on n fixed seats. Each guest either likes or does not like each of the other guests, and a guest is happier when more of the people next to them are people they like. If two guests sitting next to each other would *both* be happier after exchanging seats, they will just swap. The question is whether the host can choose a seating where no two neighbours want to swap.

The paper itself starts with a similar story about an organisation. The roles are fixed in advance, and members do not care which project they work on, only which colleagues they work with. The organisation wants to avoid two members who would both rather swap roles. This model was first studied by Bodlaender et al. (2020), and later papers by Ceylan et al., Berriaud et al. and Wilczynski looked at it further.

Why a weaker notion of stability? In hedonic games, where agents only care about who is in their group, stable outcomes often do not exist, and the same happens for seating. Berriaud et al. found many examples with no stable seating, even when the seats form a path or a cycle. In the related Schelling games, Bilò et al. showed that if agents can only swap with nearby agents, equilibria exist on more types of graphs. This paper uses the same idea and only rules out blocking pairs that sit next to each other. We think this makes sense for seating: to agree on a swap, two guests have to talk, and at a table you mostly talk to the people next to you, who are also the only people who affect your happiness. The paper also points out that it makes sense when agents only know where their neighbours are sitting.

The question. In the paper's words: *"For which classes of seat graphs is a neighborhood stable assignment guaranteed to exist under binary preferences?"*

Summary of results.

Seat graph	**Guarantee**	**Algorithm**
$K_{t,t}$, $t \geq 3$ odd	NS may not exist (Theorem 1)	–
Cycle	NS always exists (Theorem 2)	$O(n^3)$
Path	no blocking pair within distance 2 (Theorem 3)	$O(n^2)^*$
Enough leaves ($\gamma \leq$ #leaves)	NS exists (Theorem 4)	poly. if $\mathcal{P}$ is a DAG

*The paper says $O(n^3)$; we explain in Section 6 why the same algorithm runs in $O(n^2)$. NS is short for *neighbourhood stable*, γ is the directed feedback vertex set (DFVS) number of the preference graph, and DAG means directed acyclic graph.

2 Model

Definition 1 (Instance)**.** There is a set $A = \{1, \dots, n\}$ of agents, an undirected *seat graph* $G = (V, E)$ with $|V| = n$, and a directed *preference graph* $\mathcal{P}$ on A. We write $i \to j$ if (i, j) is an arc of $\mathcal{P}$, meaning agent i *approves* (likes) agent j, and $i \nrightarrow j$ if not. An *assignment* is a bijection $\pi : A \to V$.

$N_\pi(i)$ is the set of agents seated next to i. The utility of agent i is the number of neighbours it likes:

$$u_i(\pi) = \sum_{j \in N_\pi(i)} \mathbb{I}[i \to j].$$

$\pi^{i \leftrightarrow j}$ is the assignment π with i and j exchanging seats. Agent i *envies* j if $u_i(\pi^{i \leftrightarrow j}) > u_i(\pi)$, and i, j form a *blocking pair* if they envy each other. An assignment is *stable* if there is no blocking pair at all.

Definition 2 (Neighbourhood stability)**.** An assignment π is *neighbourhood stable* (NS) if there is no blocking pair sitting on adjacent vertices.

Connection to hedonic games. The paper mentions that this problem is related to hedonic games, where agents are split into groups and care about who is in their group. **[Ours]** The way we see it, the utility looks like an additively separable hedonic game with 0/1 values, except that the "group" of an agent is its set of neighbours in the seat graph and not a part of a partition.

2.1 Running example

Throughout the report we use the paper's Example 1 (its Figure 2): four agents with $a \leftrightarrow c$, $b \leftrightarrow d$, $b \to a$ and $d \to c$, seated on a path of four seats.

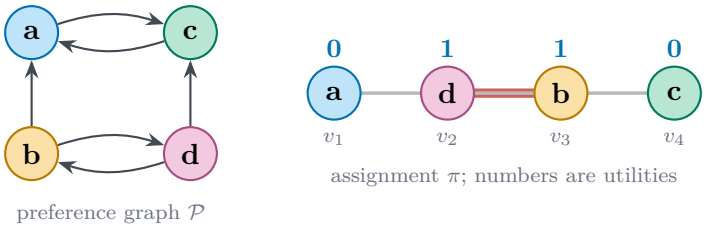


Figure 1: The running example (paper's Fig. 2). Agents b and d form an adjacent blocking pair.

In $\pi = (a, d, b, c)$, we get $u_a = u_c = 0$ and $u_b = u_d = 1$. If b and d swap, we get (a, b, d, c). Now d has lost a (whom it does not like) and gained c (whom it likes), and b has lost c and gained a. Both go up to 2, so b, d is a blocking pair sitting next to each other, and π is not NS. The new assignment (a, b, d, c) is stable, so it is also NS.

2.2 How the notions relate

Stable $\Rightarrow$ no blocking pair within distance 2 (the notion used by Berriaud et al.) $\Rightarrow$ NS. The paper also notes that checking whether a stable assignment exists is hard in general, because of a known result that this is NP-complete for the exchange-stable roommates problem.

Example 1 (NS really is weaker; paper's Example 2, taken from Massand and Simon, IJCAI 2019)**.** Take two separate triangles as the seat graph, and a directed 6-cycle $a \to b \to c \to d \to e \to f \to a$ as the preference graph. No assignment is stable: in every assignment there are two

agents at different triangles who both have utility 0, and they form a blocking pair. But every assignment is NS. Agents that sit next to each other are in the same triangle, and swapping two people within a triangle does not change anyone's neighbours. (We checked both facts for all 720 assignments.)

2.3 Why binary preferences

If preferences can take more values, things go wrong very quickly. With three non-negative values, there are four agents on a cycle with no NS assignment (the paper adapts a proof of Berriaud et al.). With values ± 1, the paper says that "it is easy to construct a counterexample of neighborhood stability on a path of length three" when the friendship graph is a cycle. **[Ours]** One example we found: three agents on a path of three seats, where each agent gives $+1$ to the next agent on a directed 3-cycle and -1 to the other one. We checked that none of the six seatings is NS. Also, two non-negative values can always be turned into binary preferences, so nothing is lost by assuming binary preferences.

3 NS assignments do not always exist

Theorem 1. *For every $n = 2t$ with $t \geq 3$ odd, there is an instance with n agents and no NS assignment.*

Construction. The seat graph is $K_{t,t}$, with two sides L and R. The agents are a top row $s_{11}, \ldots, s_{1t}$ and a bottom row $s_{21}, \ldots, s_{2t}$ (indices are taken mod t). Every agent likes exactly two agents: its *twin* in the other row ($s_{1j} \leftrightarrow s_{2j}$) and the *next agent in its own row* ($s_{ij} \to s_{i,j+1}$). So $\mathcal{P}$ is made of t two-cycles and two directed t-cycles (Figure 2).

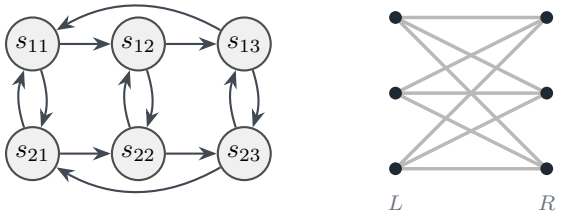


Figure 2: Theorem 1 for $t = 3$ (paper's Fig. 1): preference graph (left) and seat graph $K_{3,3}$ (right).

The main simplification. In $K_{t,t}$, the neighbours of an agent are exactly the agents on the other side. So for the utilities, the only thing that matters is which agents are on L and which are on R. Since every agent likes exactly two agents, its utility is how many of those two are on the other side. If i swaps with some j on the other side, then i's new neighbours are the agents who were on its own side, plus j. The paper uses two conditions that make i envy an agent j on the other side:

(C1) both agents that i likes are on i's own side, so $u_i = 0$. Then i envies *everyone* on the other side ($0 \to 2$).

(C2) exactly one agent that i likes is on i's own side, and $i \to j$. Then i envies j ($1 \to 2$).

Lemma 1 (ours). *In this instance, i envies j exactly when (C1) or (C2) holds.*

Proof. If i and j are on the same side, swapping does not change i's neighbours. Otherwise, let k be the number of agents that i likes on its own side. Then $u_i(\pi) = 2 - k$ and $u_i(\pi^{i \leftrightarrow j}) = k + \mathbb{I}[i \to j]$. If $k = 2$ we get $2 > 0$, which is (C1). If $k = 1$ we get $1 + \mathbb{I}[i \to j]$, which is more than 1 exactly when $i \to j$, which is (C2). If $k = 0$, i already has the maximum utility 2. □

A blocking pair needs envy in both directions, and each direction can come from either condition. Agents on opposite sides are always adjacent, so any blocking pair across the two sides breaks NS. The proof has two cases.

Case 1: some twin pair is on the same side.

1. A twin pair that is split puts one agent on each side. Both sides have t agents, so the twin pairs that are together have to be shared equally: *L and R have the same number of together pairs.* So the number of agents in together pairs is even, and since t is odd, at least one twin pair has to be split.
2. Going around the indices, there must be a together pair at some index j followed by a split pair at $j + 1$. We can rename things so that $s_{1j}, s_{2j}, s_{1,j+1} \in L$ and $s_{2,j+1} \in R$.

3. Both agents that s_{1j} likes are in L, so by (C1) it envies everyone in R.
4. If some agent in R had utility 0, it would also envy s_{1j} by (C1), and we would have a blocking pair. So every agent in R likes someone in L. Now take a together pair in R at index l. The agent s_{1l} likes s_{2l}, which is in R, so $s_{1,l+1}$ has to be in L, and in the same way $s_{2,l+1} \in L$. So every together pair in R at l is followed by a together pair in L at $l+1$.
5. This gives a one-to-one map from the together pairs in R to the together pairs in L. Both sides have the same number of together pairs, so every together pair in L is hit, including the one at j. So the pair at $j-1$ is together in R.
6. Now $s_{1,j-1} \in R$ likes $s_{2,j-1}$ (its own side) and s_{1j} (the other side), so it envies s_{1j} by (C2). With step 3, $s_{1,j-1}$ and s_{1j} form a blocking pair.

Case 2: every twin pair is split. Then L has exactly one agent from every twin pair, so at least $\lceil t/2 \rceil$ of its agents come from the same row, say the top row. These agents are on the directed t-cycle $s_{11} \to \cdots \to s_{1t} \to s_{11}$. Since t is odd, the largest set of agents on this cycle with no two next to each other has size $\lfloor t/2 \rfloor < \lceil t/2 \rceil$. So two consecutive top agents, say s_{11}, s_{12}, are both in L, and their twins s_{21}, s_{22} are in R. Now s_{11} likes s_{12} (own side) and s_{21} (other side), and s_{21} likes s_{22} (own side) and s_{11} (other side). By (C2) they envy each other.

Working through $t = 3$ [ours]. We found it useful to check the proof on $K_{3,3}$. If we do not care which side is called L, there are 10 ways to split the six agents. In Case 1, a side cannot hold two together pairs (that would be four agents), so Case 1 always means *one together pair on each side and one split pair*. There are 6 such splits. For example, take $L = \{s_{11}, s_{21}, s_{12}\}$ and $R = \{s_{22}, s_{13}, s_{23}\}$. Twins 1 are together and twins 2 are split, so $j = 1$, and s_{11} has utility 0. The only index left is $3 = j - 1$; twins 3 are together in R, and s_{13} has utility 1. Swapping s_{11} and s_{13} takes their utilities from 0 to 2 and from 1 to 2. The other 4 splits are in Case 2, for example $L = \{s_{11}, s_{12}, s_{23}\}$ and $R = \{s_{21}, s_{22}, s_{13}\}$, where s_{11} and s_{21} both go from 1 to 2 by swapping. For $t = 3$ the row is a triangle, so any two agents in it are consecutive, which makes Case 2 easy. We also checked by brute force that none of the 720 assignments is NS.

Where t odd is used. Case 1 needs a split twin pair, and Case 2 needs an odd cycle. **[Ours]** For even t this construction does not work. For $t = 4$, putting $s_{11}, s_{13}, s_{22}, s_{24}$ on L puts every agent's twin and next agent on the other side, so everyone gets the maximum utility 2 and the assignment is stable.

4 Swapping can go on forever

In *swap dynamics*, we start from any assignment and keep letting an adjacent blocking pair swap, until there is none left.

Proposition 1. *There is an instance and a starting assignment from which swap dynamics never stops, even when the seat graph is a cycle of length* 4.

The instance (the paper's Fig. 6) has $c \to a$, $a \to b$, $a \to d$, $b \to d$, $d \to c$. Figure 3 shows the four swaps. In each assignment the highlighted pair is the *only* adjacent blocking pair, so at no step is there a different swap to choose.

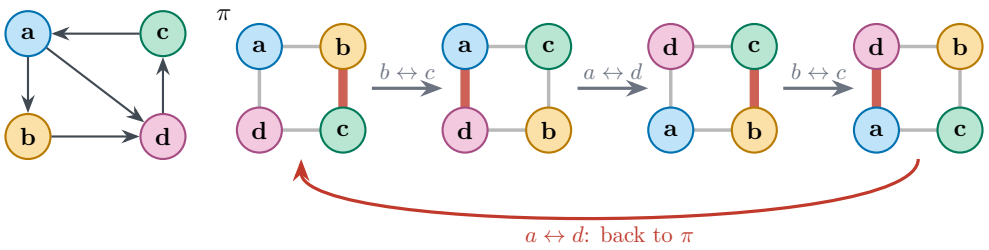


Figure 3: Proposition 1. Left: preferences. Right: the cycle of swaps on a 4-cycle seat graph; red marks the unique blocking pair. Gains: b, c go $0 \to 1$; a goes $1 \to 2$ and d goes $0 \to 1$.

What this means, and links to the course.

- The four swaps form an *improvement cycle*. From the course, a finite game has the finite improvement property (FIP) exactly when it has a generalised ordinal potential (Monderer and Shapley). If we think of each blocking swap as a "move", this means there is no function that goes up with every blocking swap on cycles.
- **[Ours]** Since the blocking pair is the only one at every step, *every* sequence of swaps from π goes around in a loop. So the dynamics is not even weakly acyclic.
- Still, this instance does have NS assignments (we counted 8 of the 24 seatings). So to show existence on cycles we need an actual construction, and that is what Theorem 2 gives.

When swapping does stop. The paper mentions that symmetric preferences ($i \to j$ exactly when $j \to i$) guarantee NS, similar to an argument of Bullinger and Suksompong. **[Ours]** Here is a short potential argument for this. Let $F(\pi)$ be the number of seat edges whose two endpoints like each other. With symmetric preferences, $u_i(\pi)$ is the number of such edges at i's seat. When i and j swap, only the edges at their two seats can change, and the edge between them (if there is one) still has the same two agents on it. So $F(\pi^{i \leftrightarrow j}) - F(\pi) = \Delta u_i + \Delta u_j > 0$ for every blocking swap, whether or not i and j are adjacent. Since $F \le |E|$, swap dynamics stops at an NS assignment. If non-adjacent blocking pairs are also allowed to swap, it stops at a *stable* assignment.

5 Cycle seat graphs

Theorem 2 (main result)**.** *If the seat graph is a cycle, an NS assignment always exists and can be found in $O(n^3)$ time.*

The paper calls this result surprising. On a cycle, ruling out blocking pairs within distance 2 can fail with five agents (Berriaud et al.), three-valued preferences can fail with four agents, and swap dynamics can loop (Proposition 1).

5.1 The swap rule

We found the following way of looking at a swap very helpful for the whole section. On a cycle C_n with $n \geq 4$, let x be at v_i and y at v_{i+1}, and let p and z be the agents at v_{i-1} and v_{i+2}. After x and y swap they are still next to each other, so x just loses p and gains z, and y loses z and gains p. So

$$x \text{ envies } y \iff x \to z \text{ and } x \not\to p, \qquad y \text{ envies } x \iff y \to p \text{ and } y \not\to z.$$

The paper's Observation 2 says that if $x \to p$ then x does not envy y, and Observation 1 says that if $y \not\to p$ or $x \not\to z$ then x, y cannot block. Both follow from the rule above. In the running example, $(x, y, p, z) = (d, b, a, c)$ satisfies all four conditions, which is why b, d block.

5.2 Splitting into two cases

Case 1 (there is a "V"). There are agents s, t, w with $s \to w \leftarrow t$, and no arc between s and t in either direction.

Case 2 (no "V"). Whenever two agents like the same agent, one of them likes the other.

The running example is in Case 1 ($b \to a \leftarrow c$, and b and c do not like each other). The instance of Proposition 1 is in Case 2: only d is liked by two agents, a and b, and $a \to b$. Checking all triples of agents tells us which case we are in, in $O(n^3)$ time.

5.3 Case 1: a greedy seating

Put s, w, t on v_1, v_2, v_3. Then for $i = 4, \ldots, n$, put on v_i an unseated agent who likes the agent on v_{i-1} if there is one, and otherwise any unseated agent.

Why it works. By the swap rule, an adjacent pair x, y (with p to the left of x) can only block if $y \to p$ and $x \not\to p$.

- Around the V: $s \not\to t$, so s does not envy w, and $t \not\to s$, so t does not envy w. Also $s \to w$ and $t \to w$, so neither s nor t envies its other neighbour (Observation 2).
- Everywhere else: take x at v_i and y at v_{i+1} with $i \geq 4$. If $x \to p$, then x does not envy y. If $x \not\to p$, then at the time x was picked, no unseated agent liked p. y was still unseated then, so $y \not\to p$, and y does not envy x.

So in every adjacent pair, either x likes the agent on its left or y does not, and that is all NS needs. Building this seating takes $O(n^2)$ time.

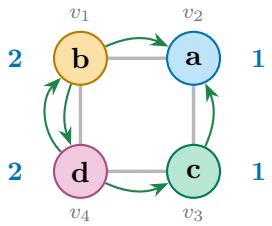


Figure 4: Case 1 on the running example with $s = b$, $w = a$, $t = c$; only d approves c, so d takes v_4. Green arrows are approvals between neighbours. Every agent attains its maximum possible utility.

5.4 Case 2: path partitions

Definition 3. A *path partition* of $\mathcal{P}$ is a set of directed paths $P_1, \ldots, P_k$ that together contain every agent exactly once. It is *minimal* if $\text{tail}(P_i) \nrightarrow \text{head}(P_j)$ for all $i \neq j$. The assignment $\Phi(P_1, \ldots, P_k)$ puts the paths one after another around the cycle (Figure 5).

Inside a path, every agent likes the agent to its right, so the only agents who might not are at the boundaries between paths. Let $R(\pi)$ be the number of agents who like their right neighbour. For a minimal partition with $k \geq 2$ paths, $R = n - k$. To get a first minimal partition, the paper keeps removing a maximal path. This is minimal because if some tail liked another path's head, one of those two paths could have been made longer when it was built.

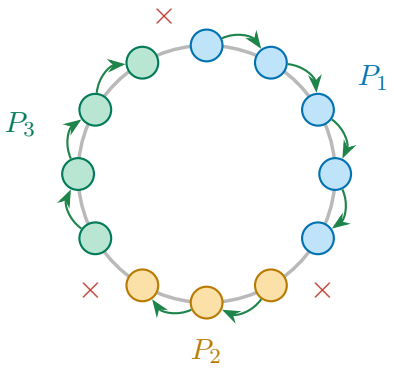


Figure 5: $\Phi(P_1, P_2, P_3)$: paths laid around the cycle (cf. paper's Fig. 4). Green: approval of the right neighbour; red crosses: path boundaries.

Claim 1 (insertion)**.** *In Case 2, if $s \notin P = s_1 \to \cdots \to s_r$ and $s \to s_r$, then either $s \to s_1$ or $s_j \to s \to s_{j+1}$ for some j.*

Idea. s and s_{r-1} both like s_r, so since there is no V, one of them likes the other. If $s_{r-1} \to s$, we put s between s_{r-1} and s_r. Otherwise $s \to s_{r-1}$, and we repeat the same argument one step back. The paper writes this as an induction on r. This is the only place where the Case 2 assumption is used.

Lemma 2. *Let $\pi = \Phi(P_1, \ldots, P_k)$ for a minimal path partition. If π has an adjacent blocking pair, then there is another minimal path partition whose assignment ρ has $R(\rho) \geq R(\pi) + 1$.*

Let the blocking pair be x, y at v_i, v_{i+1}, with p at v_{i-1} and z at v_{i+2}. The paper looks at two types.

Type I: x, y are consecutive in the same path P_r. The swap rule needs $y \nrightarrow z$. Inside a path y likes the next agent, so y must be the tail of P_r and z the head of P_{r+1}. The swap rule also gives $y \to p$ and $x \to z$. If $|P_r| \geq 3$, remove x and y from P_r. What is left ends at p, and since $y \to p$, Claim 1 lets us put y back in without changing the last agent p. Then $p \to x \to z$ joins this path onto P_{r+1} (Figure 6). If $|P_r| = 2$, put y into P_{r-1} (since $y \to p = \text{tail}(P_{r-1})$) and put x at the front of P_{r+1}. Either way there is one path fewer. After joining any other paths that can be joined, the partition is minimal again and R has gone up.

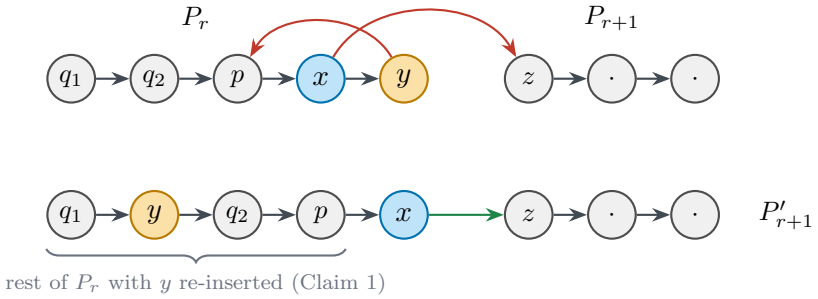


Figure 6: Type I repair with $|P_r| \geq 3$ (cf. paper's Fig. 5). Red arcs are forced by the blocking pair.

Type II: $x = \text{tail}(P_r)$ and $y = \text{head}(P_{r+1})$. The swap rule needs $y \nrightarrow z$. If $|P_{r+1}| \geq 2$, then y likes the next agent z, so P_{r+1} must be just $\{y\}$ and $z = \text{head}(P_{r+2})$. The rule also needs $x \to z$, that is $\text{tail}(P_r) \to \text{head}(P_{r+2})$, which goes against minimality. So, according to the paper, Type II cannot happen. (In Section 8 we discuss two small cases where this argument does not quite work.)

Why it stops, and how long it takes. R is at most n and goes up with every use of Lemma 2, so there are at most n repairs, and at the end there is no adjacent blocking pair. The paper says each repair takes $O(n^2)$ time but does not explain it. This is how we understand it: finding an adjacent blocking pair takes $O(n)$ (four look-ups per pair using the swap rule); the insertion in Claim 1 walks back along one path in $O(n)$; joining lists takes $O(n)$; and making the partition minimal again means checking tail–head pairs, at most $O(k^2) = O(n^2)$, where each join only creates one new tail and head to recheck. In total:

Step	Cost
Find out which case we are in (all triples)	$O(n^3)$
Case 1 greedy	$O(n^2)$
Case 2: first minimal partition	$O(n^2)$
Case 2: at most n repairs, $O(n^2)$ each	$O(n^3)$

[Ours] The case check, done by looking at triples as described, really can take about n^3 steps. When there is no V, it has to look at every pair of agents that like the same agent, and for a transitive tournament that is about $n^3/6$ pairs. The paper does not say whether a faster algorithm is possible.

Back to Proposition 1. The looping instance is in Case 2. If the first maximal path the algorithm finds is $c \to a \to b \to d$ (it contains every agent, and $d \to c$), then around the cycle every agent likes its right neighbour ($R = 4$), and the seating is NS straight away, even though swap dynamics from π never reaches an NS assignment. With a different choice of maximal

paths, the algorithm could start from the partition $\{a \to d \to c,\ b\}$ instead; we come back to this in Section 8.

Link to the course. R works like a progress measure, similar to the energy function for Hopfield networks. The difference from the potentials in PLS (Polynomial Local Search) problems is that R can only take $n+1$ values, so the number of steps is polynomial. Also, these steps are changes made by the algorithm, not swaps made by the agents.

5.5 Why distance-2 stability fails on cycles [ours]

On paths, the paper gets the stronger result of no blocking pair within distance 2 (Theorem 3). On cycles this is not possible because of the five-agent instance of Berriaud et al. The paper does not give their instance, so we looked for one ourselves with a computer search. On C_5, any two seats are at distance 1 or 2, so here distance-2 stability is the same as stability. Our instance is

$$a \to c, e; \quad b \to a, e; \quad c \to a, d; \quad d \to b, c; \quad e \to b, d,$$

which is the mutual pairs $a \leftrightarrow c$, $c \leftrightarrow d$, $b \leftrightarrow e$ plus the directed 4-cycle $a \to e \to d \to b \to a$. If we count rotations and reflections as the same, there are 12 seatings, and every one of them has a blocking pair:

Seating	Pair	Distance	Gains	NS?
$a\,b\,c\,d\,e$	c, e	2	$1 \to 2$, $1 \to 2$	yes
$a\,b\,c\,e\,d$	a, c	2	$0 \to 1$, $0 \to 1$	no
$a\,b\,d\,c\,e$	a, e	1	$1 \to 2$, $0 \to 1$	no
$a\,b\,d\,e\,c$	b, c	2	$1 \to 2$, $1 \to 2$	yes
$a\,b\,e\,c\,d$	a, e	2	$0 \to 1$, $1 \to 2$	yes
$a\,b\,e\,d\,c$	a, d	2	$1 \to 2$, $1 \to 2$	yes
$a\,c\,b\,d\,e$	b, d	1	$0 \to 1$, $1 \to 2$	no
$a\,c\,b\,e\,d$	b, d	2	$1 \to 2$, $0 \to 1$	yes
$a\,c\,d\,b\,e$	b, e	1	$1 \to 2$, $1 \to 2$	no
$a\,c\,e\,b\,d$	a, c	1	$1 \to 2$, $1 \to 2$	no
$a\,d\,b\,c\,e$	a, e	1	$1 \to 2$, $0 \to 1$	no
$a\,d\,c\,b\,e$	c, d	1	$1 \to 2$, $1 \to 2$	no

Five of these seatings are NS, which fits with Theorem 2: their only blocking pairs are at distance 2.

We think the path proof does not carry over because it uses two things a cycle does not have. On a path, the first seat is an end, so anyone who swaps into it loses a neighbour. Also, the greedy only controls each agent against the one on its left, so on a cycle nothing controls the pairs where the last seat meets the first. Case 1 above puts a V at the start exactly to make that meeting point safe for adjacent swaps; for distance 2 we would need control two seats away on both sides.

6 Path seat graphs

Theorem 3. *If the seat graph is a path, an assignment with no blocking pair within distance* 2 *exists and can be found in* $O(n^2)$ *time. (The paper says* $O(n^3)$*;* $O(n^2)$ *is our count for the same algorithm.)*

Algorithm. Put any agent on v_1. For $k = 2, \ldots, n$, put on v_k an unseated agent who likes the agent on v_{k-1} if there is one, and otherwise any unseated agent. Each of the n steps looks through at most n unseated agents, with one look-up each, so this is $O(n^2)$.

Why it works. The agents on v_2 and v_3 never envy the agent on v_1, since after such a swap their neighbours are a subset of what they had before. For $i \geq 2$, let x be the agent on v_i.

- If x likes its left neighbour, the only way a swap with v_{i+1} or v_{i+2} helps x is if it reaches utility 2. For that, x has to like the agent on v_{i+1} (who is its neighbour in both new positions), but then x already has utility 2.
- If x does not like its left neighbour, then by the way the greedy works, neither the agent on v_{i+1} nor the one on v_{i+2} likes that neighbour either (otherwise one of them would have been picked instead of x). After swapping with x, the only new neighbour they get is that left neighbour, so they gain nothing.

Running example. Starting with a: c likes a, then d likes c, then b likes d. This gives (a, c, d, b) (Figure 7), which turns out to be fully stable.

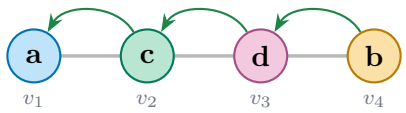


Figure 7: Theorem 3 on the running example: each new agent approves the previous one.

Background and link to the course. Berriaud et al. showed with a potential argument that swap dynamics on paths always reaches an assignment with no blocking pair within distance 2. They also gave evidence that this can take exponentially many steps, and asked whether such an assignment can be found in polynomial time. This is like the PLS situation we saw in class: the improvements have to stop at some point, but it might take exponentially long. The paper answers their question with a direct construction instead of running the dynamics.

7 A condition for general seat graphs

Definition 4. A set $X \subseteq A$ is a *directed feedback vertex set* (DFVS) if $\mathcal{P}[A \setminus X]$ has no directed cycle. The DFVS number γ is the smallest size of such a set. A *leaf* of G is a vertex with only one neighbour.

Theorem 4. *If the number of leaves of G is more than the DFVS number γ of $\mathcal{P}$, there is an NS assignment on G. When $\gamma = 0$, i.e. $\mathcal{P}$ is a DAG, a stable assignment can be found in polynomial time.*

The proof only needs a DFVS that fits on the leaves, i.e. $\gamma \leq$ #leaves (see Section 8).

Sinks. A *sink* of a directed graph is a vertex with no outgoing arcs. Every DAG has one: if you keep following arcs from any vertex, you can never come back to a vertex you already visited, so you have to end at a vertex with no outgoing arc. Removing a sink leaves another DAG, so we can order the agents of $A \setminus X$ as $s_1, s_2, \ldots$, where s_ℓ is a sink of $\mathcal{P}[A \setminus (X \cup \{s_1, \ldots, s_{\ell-1}\})]$. This means everyone that s_i likes is in X or earlier in the order.

Algorithm.

1. Put the agents of a smallest DFVS X on leaves.
2. Going through $s_1, s_2, \ldots$ in order, each agent takes the free seat that has the most already-seated neighbours it likes (ties broken any way).

Why it works. Take $i < j$. When s_i picked its seat, everyone it likes was already seated and never moves after that, and s_j's seat was still free. So s_i thought that seat was at most as good as its own, and after a swap the liked agents around that seat are the same (it does not like s_j). So s_i never envies anyone who came later, and no blocking pair at any distance can be made of two agents outside X. Now let $x \in X$ sit on a leaf, with only one neighbour y. If y swaps onto the leaf, its only neighbour becomes x, who was already its neighbour, so y does not envy x. So there is no adjacent blocking pair. If $X = \emptyset$, there is no blocking pair at all.

The choice of seat matters [ours]. Take the running example on a path $v_1v_2v_3v_4$. The two-cycles $a \leftrightarrow c$ and $b \leftrightarrow d$ do not share an agent, so $\gamma = 2$; we take $X = \{a, b\}$ on the leaves v_1, v_4. Between c and d we have $d \to c$, so c is the sink and picks first. It likes a, so it takes v_2, and d gets v_3. The result (a, c, d, b) is NS (in fact stable). If c had taken v_3 instead, we would get (a, d, c, b) with $u_c = 0$ and $u_d = 1$, and swapping c and d takes them to 1 and 2, which is an adjacent blocking pair. Also, in this example $\gamma = 2 =$ #leaves, which the proof handles but the exact statement ("more than") does not.

Consequence. Using a result of Borodin, the paper notes that planar directed graphs have DFVS number at most $\frac{3}{5}n$. So for planar preference graphs, an NS assignment exists on every seat graph where at least 60% of the vertices are leaves, for example full m-ary trees with $m \geq 3$, which have at least $\frac{m-1}{m}n$ leaves.

Link to the course [ours]. This is very similar to the existence argument for colouring games on DAGs: the agents choose in reverse topological order, sinks first, so no later choice can make an earlier one worse. One difference is that here the DAG is the *preference* graph, while in Theorem 2 the cycle is the *seat* graph.

8 Things we noticed [ours]

8.1 Two cases the proof of Lemma 2 does not cover

The Type II argument gets $\text{tail}(P_r) \to \text{head}(P_{r+2})$ and says this goes against minimality, but minimality only rules out $\text{tail}(P_i) \to \text{head}(P_j)$ for $i \neq j$. The Type I argument joins the repaired path onto P_{r+1} and uses the fact that the number of paths goes down. Both of these steps run into trouble when there are only one or two paths (indices are taken mod k).

Gap 1: Type II with $k = 2$. Here P_{r+2} is the same path as P_r, so $x \to z$ only says that P_r's tail likes its own head, which minimality allows. *Example:* in the instance of Proposition 1, the partition $\{a \to d \to c,\ b\}$ is minimal ($c \nrightarrow b$ and $b \nrightarrow a$), and Φ seats a, d, c, b around the cycle. With $(x, y, p, z) = (c, b, d, a)$ we have $c \to a$, $c \nrightarrow d$, $b \to d$ and $b \nrightarrow a$, so this is a Type II blocking pair.

Our fix. In this case P_r's tail likes its head, so P_r closes into a directed cycle, and $y \to p$ where p is the agent before x in P_r. Cut this cycle just before p and put y in front: $y \to p \to x \to z \to \cdots$ is one path with all the agents. So k goes from 2 to 1, and R goes up. In the example we get $b \to d \to c \to a$.

Gap 2: Type I with $k = 1$. Here P_{r+1} is the same path as P_r, so we cannot "join onto P_{r+1}", and the number of paths cannot go below one. *Example:* $a \to b$, $a \to c$, $b \to c$, $b \to d$, $c \to d$, $d \to a$ (this is in Case 2). The maximal path $b \to d \to a \to c$, seated around the cycle, has the Type I blocking pair $x = a$, $y = c$ (with $p = d$, $z = b$).

Our fix. Remove x and y, then put y back in using Claim 1. Since $y \nrightarrow z$, y does not go to the front, so the path still starts at z and ends at p. Adding x at the end gives a path that ends at x, and $x \to z$, so we get a directed cycle through all the agents. Now R goes from $n - 1$ to n. A seating where every agent likes its right neighbour is NS: in any adjacent pair, the right agent likes its own right neighbour, so by the swap rule it does not envy the left one. In the example we get $b \to c \to d \to a \to b$.

Testing. We wrote a program that runs Algorithm 1 the way the paper describes it, plus our two fixes, and ran it on 60,000 random instances with 4 to 9 agents (17,592 of them in Case 2). Gap 1 came up 106 times and Gap 2 came up 644 times. The paper's argument for Type II with three or more paths never failed, R went up at every repair, and every output was NS. We also checked that every one of the 4096 preference graphs on four agents has an NS assignment on C_4. So we think Theorem 2 is correct and only the written proof misses these cases.

8.2 Theorem 4: what is stated vs. what is proved

- The statement asks for the number of leaves to be *more than* γ, but the proof only uses $|X| \leq |L|$, and the paper's introduction says "upper bounded by". Our running example ($\gamma = 2 = \#\text{leaves}$) is covered by the proof but not by the statement.
- The abstract and introduction talk about a polynomial-time algorithm whenever the DFVS is at most the number of leaves, but the theorem only claims polynomial time for $\gamma = 0$. When $\gamma > 0$ the algorithm needs a small enough DFVS, and finding a smallest DFVS is NP-hard in general (Karp, 1972). The paper does not say how to find X quickly.

8.3 Running time of the path algorithm

As explained in Section 6, the greedy runs in $O(n^2)$ time, which is a better bound than the $O(n^3)$ in the paper.

8.4 Who actually finds the seating?

The motivation is about agents arranging swaps with their neighbours, but on cycles these swaps may never reach an NS assignment (Proposition 1). So the positive results tell us what a host or planner can compute, not where the agents end up on their own. On paths the swaps do eventually stop (Berriaud et al.), but possibly only after a long time.

9 Links to the course

Course topic	**Where it shows up**
Hedonic games	The utility is additively separable with 0/1 values, but over the seat neighbourhood instead of a coalition.
Potential games, FIP, generalised ordinal potentials	Proposition 1 is an improvement cycle, so there is no potential for swap dynamics on cycles. With symmetric preferences, the number of adjacent pairs who like each other is a potential.
Weakly acyclic games	From the starting assignment of Proposition 1, every improvement path loops.
Hopfield networks, progress measures	The count R in Lemma 2 goes up with each repair and has at most $n + 1$ values.
PLS	On paths, swap dynamics stops but maybe only after exponentially many steps; Theorem 3 builds the outcome directly.
Colouring games on DAGs	Theorem 4 lets sinks choose first, like the existence proof for DAGs.

10 Open questions and conclusion

The paper lists these directions for future work: other seat graphs like grids and trees, and more conditions like Theorem 4; efficiency, i.e. how much welfare is lost by requiring NS (the "price of neighbourhood stability"), or finding the best welfare among NS assignments; and preferences that are not binary, where cycles already fail with cardinal values and paths are still open.

To sum up, only letting neighbours swap turns a notion that often fails into one that always works on cycles and paths, although it can still fail on seat graphs like $K_{t,t}$. The idea we liked most is laying out path partitions of the preference graph around the cycle and fixing blocking pairs by joining paths. It gives a polynomial algorithm even though the natural swapping process can loop forever. Our checks agree with all the results in the paper, and we found two small cases in the proof of the main lemma that need the fixes described above.

A Computer checks

To make sure we were reading the paper correctly, we checked the small examples and our own claims with short Python programs that we wrote for this report. The programs store the agents, seat graphs and likes directly, and compute utilities and blocking pairs straight from the definitions in Section 2.

Exhaustive checks.

- Running example: b, d is the only adjacent blocking pair of (a, d, b, c); (a, b, d, c) is stable; the Case 1 output (b, a, c, d) on C_4 is NS; (a, c, d, b) on the path is stable; (a, d, c, b) has the adjacent blocking pair c, d.
- Proposition 1: the four swaps come back to π, the blocking pair is the only one at every step, and 8 of the 24 seatings are NS.
- Theorem 1 for $t = 3$: none of the 720 assignments on $K_{3,3}$ is NS. For $t = 4$, the assignment given in the text gives every agent utility 2.
- Example 2: all 720 assignments are NS and none is stable.
- The ± 1 example on a path of three seats: no seating is NS.
- Theorem 2 for $n = 4$: each of the 4096 preference graphs on four agents has an NS assignment on C_4.
- The five-agent instance in Section 5.5: no stable seating on C_5; 10 of the 24 seatings with a fixed on one seat are NS (5 if reflections count as the same).
- The two examples in Section 8: each has the blocking pair we describe, and the fixed assignment is NS.

Random checks.

- Theorem 3: 20,000 random instances with 3 to 8 agents; the greedy output never had a blocking pair within distance 2.
- Theorem 2: Algorithm 1 as in the paper, plus our two fixes from Section 8, on 60,000 random instances with 4 to 9 agents. At every repair the program checked the things the proof relies on (the type of the blocking pair, the arcs that the swap rule forces, that Claim 1 works, and that R goes up), and all outputs were NS.

These checks support the results but they are not proofs; for bigger instances we rely on the arguments in the text.

References

[1] H. Aziz, G. Lisowski, M. Suzuki and J. Vollen. Neighborhood stability in assignments on graphs. arXiv:2407.05240, 2024.

[2] D. Berriaud, A. Constantinescu and R. Wattenhofer. Stable dinner party seating arrangements. WINE 2023; arXiv:2305.09549.

[3] H. L. Bodlaender, T. Hanaka, L. Jaffke, H. Ono, Y. Otachi and T. C. van der Zanden. Hedonic seat arrangement problems. AAMAS 2020.

[4] D. Bilò, V. Bilò, P. Lenzner and L. Molitor. Topological influence and locality in swap Schelling games. Autonomous Agents and Multi-Agent Systems 36(2), 2022.

[5] M. Bullinger and W. Suksompong. Topological distance games. Theoretical Computer Science 981, 2024.

[6] S. Massand and S. Simon. Graphical one-sided markets. IJCAI 2019.

[7] O. V. Borodin. On acyclic colorings of planar graphs. Discrete Mathematics 25(3), 1979.

[8] D. Monderer and L. S. Shapley. Potential games. Games and Economic Behavior 14(1), 1996.

[9] R. M. Karp. Reducibility among combinatorial problems. In Complexity of Computer Computations, 1972.